

04

Manual de prompts, agentes y ahorro de tokens

Dirigir a la IA para que no falle y no te cueste una fortuna

Para quién es esto y cómo leerlo

Tú construyes sistemas de reservas para negocios locales: una barbería, un consultorio dental, un taller mecánico, un estudio de uñas. La base técnica es siempre la misma: **Next.js + TypeScript + Tailwind + Supabase + Vercel**. Y no necesitas saber programar para entregarlo, porque no eres tú quien escribe el código: es Claude Code.

Pero eso mueve el trabajo de lugar. Tu habilidad ya no es "escribir una función". Tu habilidad es **dirigir**. Eres el que define qué se construye, con qué reglas, en qué orden, y cómo se sabe que quedó terminado. La IA es un desarrollador muy rápido, con memoria corta, sin contexto de tu negocio y sin miedo a inventar. Tu trabajo es quitarle esas tres desventajas.

Este manual está ordenado de lo conceptual a lo copiable:

Parte	Qué aprendes
1	Por qué la IA falla (las 4 causas reales)
2	Anatomía de un prompt que no falla
3	CLAUDE.md: el cerebro del proyecto
4	Skills: automatizar tu propio criterio
5	Cómo escribir el <code>description</code> para que la skill se active
6	Subagentes: 4 listos para copiar
7	Orquestación: el flujo completo de un sistema de reservas
8	Ahorro de tokens: 18 tácticas con antes y después
9	Protocolo de depuración en 6 pasos
10	Biblioteca de 20 prompts listos
11	Los 12 errores más caros

Lee las partes 1 y 2 completas. Las demás úsalas como referencia mientras construyes.

PARTE 1 — El modelo mental: por qué la IA falla

Cuando la IA te entrega algo que no sirve, casi nunca es porque "no sea capaz". Es porque tú le diste un problema mal planteado. Después de cientos de sesiones construyendo sistemas de reservas, todos los fracasos caen en cuatro cubetas.

Causa 1: Contexto ausente

La IA no sabe nada que no le hayas dicho o que no pueda leer en tu proyecto. No sabe que la barbería abre de 9 a 20 con corte de 45 minutos. No sabe que el cliente quiere cobrar anticipo. No sabe que ya tienes una tabla `bookings` con una columna `status`.

Cuando falta contexto, la IA **rellena el hueco inventando** algo razonable. Y algo razonable en abstracto suele ser un desastre en tu caso concreto.

Síntoma: te entrega código correcto que resuelve un problema que no era el tuyo.

Corrección: el contexto que se repite va en `CLAUDE.md`. El contexto de la tarea concreta va en el prompt. El contexto largo y ocasional va en un archivo que le pides leer por ruta exacta.

Causa 2: Instrucción ambigua

"Haz el calendario más bonito" no es una instrucción, es un deseo. "Que funcione bien en móvil" tampoco. La IA va a tomar una decisión por ti, y como no le diste criterio, va a tomar la suya.

Síntoma: hace algo, tú dices "no, así no", y entras en un ciclo de tres o cuatro intentos donde cada uno cuesta tokens y ninguno converge.

Corrección: convierte adjetivos en hechos verificables. "Bonito" no. "Los slots ocupados en gris `#9CA3AF` sin cursor pointer, los libres en blanco con borde, el seleccionado con fondo negro y texto blanco" sí.

Causa 3: Tarea demasiado grande

"Hazme el sistema de reservas completo" es un encargo que ningún desarrollador humano aceptaría sin desglosar. La IA sí lo acepta, y ahí está el problema: lo intenta. Genera veinte archivos a la vez, la mitad con supuestos equivocados, y cuando algo falla no sabes qué pieza es la culpable.

Síntoma: mucho código generado de golpe, nada funciona del todo, y depurarlo cuesta más que haberlo hecho por partes.

Corrección: trabaja en **tareas cerradas**. Una tarea cerrada es la que se puede verificar sola: "crea la tabla `services` con estas columnas y su migración" es cerrada. "Monta el backend" no lo es. Regla práctica: si no puedes decir en una frase cómo comprobarás que quedó bien, la tarea es demasiado grande.

Causa 4: Falta de criterio de "terminado"

Sin criterio de aceptación, la IA se detiene cuando *le parece*. A veces se detiene demasiado pronto (deja un `TODO`), a veces demasiado tarde (te refactoriza medio proyecto que no habías pedido).

Síntoma: "ya está listo" pero no está listo. O está listo y además cambió cinco cosas que funcionaban.

Corrección: cierra todo prompt con una condición de éxito comprobable. "Terminado significa: `npm run build` pasa sin errores, la ruta `/reservar` carga, y al intentar reservar un horario ya tomado sale el mensaje 'Ese horario ya no está disponible'."

Tabla resumen

Causa	Cómo se ve	Corrección en una línea
Contexto ausente	Resuelve el problema equivocado	Dale los datos: reglas de negocio, esquema, rutas
Instrucción ambigua	Ciclo de "no, así no"	Cambia adjetivos por hechos verificables
Tarea demasiado grande	Mucho código, nada funciona	Divide en tareas verificables una a una
Sin criterio de terminado	"Ya está" y no está	Define la condición de éxito antes de empezar

Casi todo lo que sigue en este manual es una herramienta para atacar una de estas cuatro causas de forma sistemática.

PARTE 2 — Anatomía de un prompt que no falla

Un buen prompt tiene seis componentes. No siempre necesitas los seis, pero cuando algo sale mal, casi siempre falta uno de estos.

#	Componente	Pregunta que responde
1	Objetivo	¿Qué quiero que exista al final?
2	Contexto	¿Qué debe saber para no inventar?
3	Restricciones	¿Con qué tecnología, versiones y límites?
4	Formato de salida	¿Qué me entrega y cómo?
5	Criterio de aceptación	¿Cómo sabemos que quedó bien?
6	Qué NO hacer	¿Qué está prohibido tocar o asumir?

El componente 6 es el que más gente omite y el que más dinero ahorra. Sin él, la IA "aprovecha para" reorganizar carpetas, cambiar librerías o reescribir archivos que funcionaban.

Comparación 1: la tabla de reservas

Mal:

```
Créame la tabla de reservas en Supabase.
```

Bien:

OBJETIVO: crear la tabla `bookings` en Supabase con su migración SQL.

CONTEXT0: sistema de reservas para una barbería con 3 barberos.

Ya existen las tablas `services` (id, name, duration_min, price_cents) y `staff` (id, full_name, active).

RESTRICCIONES:

- Postgres en Supabase. Usa uuid como PK con gen_random_uuid().
- Todas las horas se guardan en timestamptz, en UTC.
- Campos: id, service_id (FK), staff_id (FK), customer_name, customer_phone, starts_at, ends_at, status, created_at.
- `status` debe ser un enum: pending, confirmed, cancelled, no_show.

FORMATO DE SALIDA: un único archivo SQL de migración en supabase/migrations/ con nombre por fecha. Nada más.

ACEPTACIÓN: el SQL corre sin errores en un Postgres limpio que ya tenga services y staff, y crea índice sobre (staff_id, starts_at).

NO HAGAS: no toques las tablas services ni staff, no crees políticas RLS todavía (van en otra tarea), no generes código de la app.

La diferencia no es que el segundo sea más largo. Es que el segundo **no tiene huecos que la IA tenga que rellenar inventando**.

Comparación 2: el calendario de disponibilidad

Mal:

Hazme un calendario donde el cliente elija la hora.

Bien:

OBJETIVO: componente React `<AvailabilityCalendar />` que muestre los horarios disponibles de un barbero para una fecha, y permita elegir uno.

CONTEXTO: Next.js App Router + TypeScript + Tailwind. Los datos vienen de la función existente `getAvailableSlots(staffId, date)` que ya está en `lib/availability.ts` y devuelve `string[]` con horas ISO.

RESTRICCIONES:

- Props: `staffId: string`, `date: Date`, `onSelect: (iso: string) => void`.
- Solo Tailwind, sin librerías de calendario nuevas.
- Slots ocupados no se renderizan; solo se muestran los libres.
- Debe funcionar desde 360px de ancho: grid de 3 columnas en móvil, 6 en escritorio.

FORMATO DE SALIDA: un solo archivo `components/booking/AvailabilityCalendar.tsx`.

ACEPTACIÓN: si `getAvailableSlots` devuelve `[]` se muestra el texto "No hay horarios disponibles para este día". Al hacer clic en un slot se llama `onSelect` con el ISO exacto y ese slot queda con fondo negro y texto blanco.

NO HAGAS: no instales dependencias, no modifiques `lib/availability.ts`, no metas lógica de `fetch` dentro del componente.

Comparación 3: evitar la doble reserva

Mal:

Asegúrate de que no se puedan hacer dos reservas a la misma hora.

Bien:

OBJETIVO: impedir a nivel de base de datos que dos reservas activas del mismo barbero se solapen en el tiempo.

CONTEXTO: tabla `bookings` con `staff_id`, `starts_at`, `ends_at`, `status`. Una reserva cancelada o `no_show` NO bloquea el horario. Hoy la validación solo existe en el cliente, y eso no sirve: dos personas pueden reservar el mismo slot en el mismo segundo.

RESTRICCIONES:

- La garantía debe estar en Postgres, no en la app.
- Considera usar la extensión `btree_gist` con una constraint `EXCLUDE` sobre `tstzrange(starts_at, ends_at)`, filtrada por `status` activo.
- Si eso no es viable, propón la alternativa y explica por qué antes de escribir código.

FORMATO DE SALIDA: migración SQL + explicación de 5 líneas de qué error verá la app cuando la constraint se dispare.

ACEPTACIÓN: insertar dos reservas confirmadas solapadas del mismo `staff_id` falla; insertar la segunda con `status cancelled` funciona; dos barberos distintos a la misma hora funcionan.

NO HAGAS: no muevas la validación al frontend, no borres datos existentes, no cambies el tipo de las columnas.

Comparación 4: adaptar el proyecto a otro nicho

Mal:

Cámbialo para que sirva para una clínica dental.

Bien:

OBJETIVO: adaptar el sistema de reservas de barbería a clínica dental, cambiando solo lo necesario.

CONTEXTO: la estructura de datos es compatible (servicio, profesional, horario). Las diferencias reales son cuatro:

1. "barbero" pasa a llamarse "doctor" en toda la interfaz visible.
2. Las citas duran 30 o 60 min según el tratamiento (no 45 fijo).
3. Hay que pedir motivo de consulta (texto libre, máx 300 caracteres).
4. Aviso legal de datos de salud en el formulario.

RESTRICCIONES:

- Solo cambios de UI, textos y un campo nuevo. No rediseñes el esquema más allá de agregar `notes` a bookings.
- Los textos visibles al usuario deben quedar en es-MX.

FORMATO DE SALIDA: primero dame la lista de archivos que vas a tocar y qué cambio harás en cada uno. Espera mi OK antes de editar nada.

ACEPTACIÓN: tras el cambio, `npm run build` pasa y no queda la palabra "barbero" en ningún archivo dentro de app/ ni components/.

NO HAGAS: no renombres tablas ni columnas de la base de datos, no cambies el sistema de autenticación, no toques las políticas RLS.

Ese "dame la lista antes de editar" es una de las técnicas más rentables del manual: convierte un cambio grande y ciego en un plan que tú apruebas, y evita rehacer trabajo.

PARTE 3 — CLAUDE.md: el cerebro del proyecto

`CLAUDE.md` es un archivo markdown en la raíz de tu proyecto que Claude Code lee como contexto permanente. Es donde vive lo que **siempre** es verdad en este proyecto.

Piénsalo así: si tuvieras que dejarle una nota a un desarrollador nuevo que empieza mañana y no vas a poder contestarle preguntas, ¿qué le escribirías? Eso va en `CLAUDE.md`.

Qué SÍ va dentro

- **Qué es el proyecto**, en tres líneas. Para quién, qué resuelve.
- **Stack y versiones**: Next.js con App Router, TypeScript estricto, Tailwind, Supabase, despliegue en Vercel.
- **Mapa de carpetas**: dónde vive cada cosa, con rutas reales.
- **Reglas de negocio numeradas** (la parte más valiosa).
- **Convenciones de código**: cómo nombrar archivos, dónde van los tipos, si se usan Server Components por defecto.
- **Comandos del proyecto**: cómo se corre, cómo se construye, cómo se aplican migraciones.
- **Cosas prohibidas**: qué no se toca nunca sin permiso.

Qué NO va dentro

- Documentación larga de librerías. La IA ya conoce Next.js; no le pegues el manual.
- Procedimientos largos que solo se usan de vez en cuando. **Eso es una skill.**
- Historial de decisiones, changelogs, notas de reuniones.
- Código de ejemplo extenso. Una referencia a un archivo real vale más.
- Cosas que cambian cada semana. Se desactualizan y la IA obedece información vieja.

El criterio es simple: **si aplica al 90% de los turnos, va en `CLAUDE.md`. Si aplica a un tipo de tarea concreta, va en una skill.**

Por qué las reglas de negocio deben ser explícitas y numeradas

Las reglas de negocio son el conocimiento que no está en ningún lado del código, y son justo lo que la IA no puede adivinar. Numerarlas tiene tres ventajas concretas:

1. Puedes **referenciarlas en un prompt**: "respetar la regla 4" en vez de reexplicarla. Eso ahorra tokens en cada mensaje.
2. Puedes **auditarlas**: "revisa el archivo X y dime qué reglas de negocio del CLAUDE.md no se están cumpliendo".
3. Fuerzan a que **tú** las tengas claras. La mitad de los bugs de un sistema de reservas nacen de una regla que el dueño del negocio nunca dijo en voz alta.

Plantilla de CLAUDE.md

```
# Sistema de reservas – Barbería El Corte

## Qué es
App web de reservas para una barbería con 3 barberos. El cliente
elige servicio, barbero, día y hora. El negocio ve todo en un panel.

## Stack
- Next.js (App Router) + TypeScript en modo estricto
- Tailwind para todo el estilo. Sin librerías de UI adicionales.
- Supabase (Postgres + Auth) como backend
- Despliegue en Vercel

## Mapa del proyecto
- app/(public)/      páginas del cliente final
- app/(admin)/       panel del negocio, requiere sesión
- components/booking/ componentes del flujo de reserva
- lib/supabase/       clientes de Supabase (server y browser)
- lib/availability.ts cálculo de horarios disponibles
- supabase/migrations/ migraciones SQL, una por cambio

## Reglas de negocio
1. El horario de atención es 09:00-20:00, lunes a sábado. Domingo cerrado.
2. Cada servicio tiene su propia duración; no existe una duración fija.
3. Un barbero no puede tener dos reservas activas solapadas.
   Cancelada y no_show NO ocupan el horario.
4. No se aceptan reservas con menos de 2 horas de anticipación.
5. No se aceptan reservas a más de 30 días vista.
6. El teléfono del cliente es obligatorio; el email es opcional.
7. Cancelar es posible hasta 4 horas antes de la cita.
8. Todas las horas se guardan en UTC (timestampz) y se muestran
   en America/Mexico_City.

## Convenciones
- Server Components por defecto. "use client" solo si hay interacción.
- Los tipos compartidos van en types/index.ts.
- Los textos visibles al usuario van en español de México.
- Nunca se usa la service_role key en código del navegador.

## Comandos
- Desarrollo: npm run dev
- Build de verificación: npm run build
- Migraciones: se aplican con el CLI de Supabase

## Prohibido sin permiso explícito
- Instalar dependencias nuevas
- Renombrar tablas o columnas existentes
- Modificar las políticas RLS
- Borrar archivos
```

El error de meter en CLAUDE.md lo que debería ser una skill

Este es el error más común y el más caro, porque **cuesta tokens en cada uno de tus mensajes**.

CLAUDE.md se carga siempre. Si le metes un procedimiento de 400 líneas sobre cómo hacer una auditoría de seguridad pre-entrega, estás pagando esas 400 líneas en cada turno de cada sesión, incluso cuando estás cambiando un color de botón.

Una skill, en cambio, se carga **solo cuando hace falta**. El procedimiento de auditoría vive ahí y solo entra al contexto el día que auditas.

Si es...	Va en
Reglas de negocio del proyecto	CLAUDE.md
Stack y convenciones	CLAUDE.md
Mapa de carpetas	CLAUDE.md
Procedimiento de auditoría de seguridad	Skill
Checklist de despliegue a Vercel	Skill
Proceso para adaptar el proyecto a otro nicho	Skill
Formato exacto de los textos legales	Skill

Un CLAUDE.md sano cabe en dos pantallas. Si el tuyo tiene 500 líneas, hay tres skills escondidas ahí dentro.

PARTE 4 — Skills: automatizar tu propio criterio

Qué es una skill

Una skill es una carpeta con un archivo `SKILL.md` dentro. Ese archivo contiene un procedimiento que tú ejecutas seguido, escrito una vez, para no tener que repetirlo nunca más.

Ubicaciones:

Alcance	Ruta
Personal (todos tus proyectos)	<code>~/.claude/skills/<nombre>/SKILL.md</code>
De proyecto (solo este repo)	<code>.claude/skills/<nombre>/SKILL.md</code>

Dato importante: el nombre del comando sale del **nombre de la carpeta**. Si la carpeta se llama `auditoria-seguridad`, el comando es ese. En skills personales y de proyecto, el campo `name` del frontmatter es solo una etiqueta para el listado; no cambia cómo se invoca.

Cuándo crear una skill

Crea una skill cuando se cumpla alguna de estas:

- Has escrito el mismo bloque de instrucciones **tres veces** en distintos chats.
- Tienes un procedimiento con **pasos en orden** que si se saltan da resultados malos.
- Estás a punto de agregar más de 30 líneas a `CLAUDE.md` sobre un tema puntual.
- Quieres que un proceso quede igual aunque lo ejecutes dentro de seis meses.
- Vendes el mismo tipo de proyecto una y otra vez, cambiando el nicho.

Ese último punto es el corazón del negocio. Si entregas sistemas de reservas a diez negocios distintos, cada paso repetible que conviertas en skill es margen que recuperas.

Estructura de carpetas

```
.claude/skills/entregar-proyecto/  
├── SKILL.md           <- corto: el procedimiento  
├── references/  
│   ├── checklist-seguridad.md <- largo, se lee solo si hace falta  
│   ├── textos-legales.md  
│   └── variables-entorno.md  
└── scripts/  
    └── verificar-build.sh
```

Frontmatter: campos reales disponibles

El frontmatter va entre `---` al inicio del `SKILL.md`. **Todos los campos son opcionales**; se recomienda al menos `description`.

Campo	Para qué sirve
<code>name</code>	Etiqueta de listado (el comando sale de la carpeta)
<code>description</code>	Cuándo debe usarse la skill. El campo más importante
<code>when_to_use</code>	Detalle adicional de las situaciones de uso
<code>argument-hint</code>	Pista de qué argumentos espera
<code>arguments</code>	Definición de los argumentos
<code>disable-model-invocation</code>	Evita que el modelo la invoque por su cuenta
<code>user-invocable</code>	Si el usuario puede invocarla directamente
<code>allowed-tools</code>	Herramientas permitidas durante la skill
<code>disallowed-tools</code>	Herramientas bloqueadas durante la skill
<code>model</code>	Modelo a usar para esta skill
<code>effort</code>	Nivel de esfuerzo
<code>context</code>	Con valor <code>fork</code> corre en un subagente
<code>agent</code>	Agente asociado
<code>background</code>	Ejecución en segundo plano
<code>hooks</code>	Hooks asociados
<code>paths</code>	Rutas relevantes
<code>shell</code>	Configuración de shell

Límite que debes conocer: la combinación de `description` + `when_to_use` se trunca a **1.536 caracteres** en el listado. Todo lo que pase de ahí no se ve. Por eso el caso de uso principal va **primero**, no al final.

Sustituciones disponibles en el cuerpo

Sustitución	Qué inserta
<code>\${CLAUDE_SKILL_DIR}</code>	La carpeta de la skill
<code>\${CLAUDE_PROJECT_DIR}</code>	La raíz del proyecto
<code>\$ARGUMENTS</code>	Todos los argumentos
<code>\$0 , \$1 , \$2 ...</code>	Argumentos posicionales

Divulgación progresiva: el principio que ahorra más tokens

Aquí está el detalle que casi nadie entiende y que te va a ahorrar dinero real:

El cuerpo del `SKILL.md` permanece en contexto durante los turnos siguientes una vez cargado.

No se lee y se olvida. Se queda. Cada línea que escribas ahí la estás pagando de forma **recurrente** durante el resto de la sesión.

Consecuencia práctica: el `SKILL.md` debe ser **corto y ser un índice de decisiones**, no una enciclopedia. Todo lo largo se mueve a `references/` y se lee **solo cuando hace falta**.

Eso es divulgación progresiva: primero lo mínimo, y el detalle solo bajo demanda.

Va en el <code>SKILL.md</code>	Va en <code>references/</code>
Los pasos, en orden	El checklist de 60 puntos
Las decisiones clave	Los textos legales completos
Cuándo leer cada referencia	Las tablas de configuración largas
Los criterios de aceptación	Ejemplos extensos de código

Regla de dedo: si tu `SKILL.md` pasa de 100 líneas, algo de ahí pertenece a `references/`.

Plantilla en blanco de SKILL.md

```
---
name: nombre-para-el-listado
description: "[Caso de uso principal primero]. Úsala cuando el usuario
  diga "[frase real 1]", "[frase real 2]", "[frase real 3]". También
  cubre [caso secundario]. No la uses para [caso que NO cubre].
when_to_use: "[Detalle de situaciones adicionales, si hace falta]"
argument-hint: "[qué esperas como argumento]"
---

# [Título del procedimiento]

## Qué logra esto
[Una o dos líneas. Qué existe al terminar.]

## Antes de empezar, confirma
- [Dato 1 que necesitas del usuario]
- [Dato 2]
Si falta alguno, pregúntalo antes de ejecutar nada.

## Pasos

### 1. [Nombre del paso]
[Instrucción concreta.]

### 2. [Nombre del paso]
[Instrucción concreta.]
Si necesitas el detalle completo, lee
${CLAUDE_SKILL_DIR}/references/[archivo].md

### 3. [Nombre del paso]
[Instrucción concreta.]

## Criterio de terminado
- [ ] [Condición verificable 1]
- [ ] [Condición verificable 2]

## Qué NO hacer
- [Prohibición 1]
- [Prohibición 2]
```

Ejemplo completo y funcional

Carpeta: `.claude/skills/nuevo-nicho/`

`.claude/skills/nuevo-nicho/SKILL.md`


```

---
name: nuevo-nicho
description: Adapta el sistema de reservas base a un nicho nuevo
  (clínica dental, taller mecánico, estudio de uñas, spa, veterinaria,
  consultorio). Úsala cuando el usuario diga "adáptalo para una
  clínica", "ahora es para un taller", "cámbialo de nicho", "clona el
  proyecto para otro cliente", "esto es para un spa", "adaptalo a
  veterinaria". Cubre renombrar la terminología visible, ajustar
  duraciones de servicio, campos extra del formulario y textos
  legales del rubro. No la uses para cambios de diseño ni para
  agregar funciones nuevas al producto.
when_to_use: Cuando ya existe un proyecto de reservas funcionando y
  hay que entregárselo a un cliente de otro rubro.
argument-hint: "[nicho del nuevo cliente]"
---

# Adaptar el sistema de reservas a un nicho nuevo

## Qué logra esto
Deja el proyecto listo para un cliente de otro rubro, cambiando solo
terminología, duraciones, campos y textos. Sin tocar el esquema base.

## Antes de empezar, confirma con el usuario
- Nicho exacto y nombre comercial del negocio
- Cómo se llama el profesional en ese rubro (doctor, técnico, estilista)
- Lista de servicios con su duración en minutos y precio
- Horario de atención y días cerrados
- Si necesita algún campo extra en el formulario de reserva
Si falta algún dato, pregúntalo. No inventes ninguno.

## Pasos

### 1. Inventario de cambios
Lista los archivos que contienen terminología del nicho anterior.
Devuelve la lista y espera aprobación antes de editar.

### 2. Terminología visible
Reemplaza solo textos visibles al usuario en app/ y components/.
No renombres tablas, columnas, variables ni rutas.

### 3. Servicios y duraciones
Genera el SQL de inserción para la tabla `services` con los datos
confirmados. Una migración nueva; no edites migraciones anteriores.

### 4. Reglas de negocio
Actualiza la sección "Reglas de negocio" de CLAUDE.md con el horario
y las restricciones del nuevo cliente. Mantén la numeración.

### 5. Textos legales
Lee ${CLAUDE_SKILL_DIR}/references/textos-legales.md y usa la
plantilla del rubro correspondiente.

### 6. Verificación
Ejecuta `npm run build`. Debe pasar sin errores ni advertencias
de tipos.

## Criterio de terminado
- [ ] `npm run build` pasa limpio

```

- [] No queda terminología del nicho anterior en app/ ni components/
- [] La tabla services tiene los servicios reales del cliente
- [] CLAUDE.md refleja el horario y las reglas del cliente nuevo
- [] Las páginas legales nombran al negocio correcto

Qué NO hacer

- No renombres tablas ni columnas
- No cambies el sistema de autenticación
- No instales dependencias nuevas
- No modifiques las políticas RLS
- No borres migraciones existentes

Fíjate en el tamaño: cabe en una pantalla y media. Los textos legales completos, que ocuparían 300 líneas, están en `references/textos-legales.md` y solo entran al contexto en el paso 5.

PARTE 5 — Cómo escribir el `description` para que la skill se active

Una skill que no se activa cuando debe es una skill que no existe. El `description` es lo único que se ve antes de cargarla, así que es lo que decide si se usa.

Las cuatro reglas

Regla 1: el caso de uso principal va primero. El listado se trunca a 1.536 caracteres contando `description` + `when_to_use`. Si tu caso principal está en la línea 12, puede no llegar.

Regla 2: incluye frases disparadoras reales. No describas la skill como la describirías en una junta. Escribe **literalmente lo que tú tecleas** cuando la necesitas: "adáptalo para una clínica", "revísalo antes de entregar", "prepáralo para producción".

Regla 3: incluye errores de escritura comunes. Tú escribes rápido y sin acentos. Si sueles escribir "adaptalo", "revisame", "seguridad rls", ponlo. La skill debe activarse con lo que realmente escribes, no con lo que deberías escribir.

Regla 4: di también qué NO cubre. Una frontera clara evita que se active en la tarea equivocada, lo cual cuesta tokens y confunde el resultado.

Tres descriptions malas y sus correcciones

Mala 1

```
description: Skill de seguridad.
```

Problema: no dice cuándo usarla, ni con qué palabras, ni sobre qué. Nunca se va a activar sola.

Corregida:

```
description: Audita la seguridad del sistema de reservas antes de entregarlo a un cliente – políticas RLS de Supabase, claves expuestas, validación en servidor y control de acceso al panel de administración. Úsala cuando el usuario diga "revisa la seguridad", "revisame la seguridad antes de entregar", "está seguro esto", "chequea las RLS", "seguridad rls", "auditoria antes de entregar", "puedo publicarlo ya", "revision previa a produccion". No la uses para optimizar rendimiento ni para revisar estilo de código.
```

Mala 2

```
description: Esta skill implementa un flujo completo y robusto de
verificación de calidad que abarca múltiples dimensiones del
desarrollo moderno, incluyendo pero no limitándose a la
comprobación de la coherencia arquitectónica, la revisión de
patrones de diseño aplicados, la validación de la experiencia de
usuario, la evaluación del rendimiento percibido, el análisis de
la accesibilidad, la verificación de la compatibilidad entre
navegadores y, finalmente, la preparación del despliegue.
```

Problema: 500 caracteres de palabrería antes de decir nada útil, cero frases disparadoras, y el caso concreto ("preparación del despliegue") queda al final. Además compite consigo misma: abarca tanto que no se sabe cuándo aplica.

Corregida:

```
description: Prepara el proyecto para desplegar en Vercel – variables
de entorno, build de verificación, dominio y comprobación post-
despliegue. Úsala cuando el usuario diga "vamos a desplegar",
"súbelo a Vercel", "públicalo", "ponlo en producción", "deploy",
"ya está listo para subir", "conectar el dominio". No la uses para
auditar seguridad (eso es la skill de auditoría) ni para arreglar
errores de build.
```

Mala 3

```
description: Crea componentes de UI.
when_to_use: Cuando se necesiten componentes.
```

Problema: demasiado genérica. Se va a activar en tareas donde no aporta nada, quemando contexto, y además choca con cualquier otra skill de frontend.

Corregida:

```
description: Construye los componentes del flujo de reserva del
cliente final – selector de servicio, selector de profesional,
calendario de disponibilidad, formulario de datos y pantalla de
confirmación – respetando el sistema visual del proyecto. Úsala
cuando el usuario diga "hazme el paso de elegir hora", "el
calendario de reservas", "la pantalla de confirmación", "el
formulario del cliente", "el flujo de reserva", "componente de
disponibilidad", "calendario disponibilidad". No la uses para el
panel de administración ni para páginas de marketing.
```

Prueba rápida antes de dar por buena una description

Léela y contesta: ¿un lector que no conoce la skill sabría, en los primeros 15 segundos, en qué situación exacta usarla y en cuál no? Si la respuesta es no, reescríbela empezando por el caso de uso.

PARTE 6 — Subagentes

Qué son y por qué ahorran contexto

Un subagente es un asistente especializado que **corre en su propia ventana de contexto** y solo devuelve **su resultado final** a la conversación principal.

Eso es lo importante. Si le pides a un subagente que explore veinte archivos buscando dónde se valida el teléfono, el subagente lee esos veinte archivos en **su** contexto y te devuelve tres líneas. Tu conversación principal recibe tres líneas, no veinte archivos.

Es la diferencia entre mandar a alguien a la biblioteca y que te traiga la respuesta, o traerte la biblioteca a la sala.

Formato

Alcance	Ruta
Personal	<code>~/ .claude/agents/<nombre>.md</code>
De proyecto	<code>.claude/agents/<nombre>.md</code>

Frontmatter disponible: `name` , `description` , `tools` (lista separada por comas), `model` . El **cuerpo del archivo es el system prompt del subagente**. Los nombres deben ser únicos.

Cuándo usar un subagente

Úsalo cuando	No lo uses cuando
Hay que explorar/leer mucho y devolver poco	La tarea es un cambio pequeño y directo
Quieres una revisión con criterio independiente	Necesitas ir y venir contigo en cada paso
La tarea es repetible y bien definida	El objetivo aún no está claro
Quieres proteger tu contexto principal	Estás en medio de una edición delicada

Subagente 1: revisor de seguridad

`.claude/agents/revisor-seguridad.md`

```
---
name: revisor-seguridad
description: Revisa la seguridad de un sistema de reservas construido
  con Next.js y Supabase antes de entregarlo al cliente. Busca claves
  expuestas, tablas sin RLS, validación que solo existe en el cliente
  y rutas de administración sin protección. Devuelve una lista de
  hallazgos priorizados.
tools: Read, Grep, Glob
model: inherit
---
```

Eres un revisor de seguridad especializado en aplicaciones web de reservas hechas con Next.js (App Router), TypeScript y Supabase. Tu único trabajo es encontrar problemas y reportarlos. No corriges nada, no editas archivos, no ejecutas comandos.

Qué revisas, en este orden

1. SECRETOS EXPUESTOS

- Busca la `service_role` key de Supabase en cualquier archivo que pueda llegar al navegador: componentes con `"use client"`, archivos en `public/`, cualquier variable con prefijo `NEXT_PUBLIC_`.
- Busca claves, tokens o contraseñas escritas directamente en el código en lugar de leerse de variables de entorno.
- Verifica que `.env` y `.env.local` estén en `.gitignore`.

2. ROW LEVEL SECURITY

- Para cada tabla que encuentres en `supabase/migrations/`, confirma que tiene RLS habilitado.
- Marca como hallazgo grave cualquier política que use ``true`` como condición para `SELECT`, `UPDATE` o `DELETE` sin filtro de usuario.
- Revisa que las reservas de un cliente no sean legibles por otro.

3. VALIDACIÓN EN SERVIDOR

- Toda regla de negocio (anticipación mínima, horario de atención, solapamiento, duración) debe existir en el servidor o en la base de datos, no solo en el formulario.
- Reporta cada regla que solo esté validada en el cliente.

4. CONTROL DE ACCESO

- Las rutas del panel de administración deben verificar sesión en el servidor antes de renderizar datos.
- Reporta cualquier página de admin que solo esconda la interfaz sin verificar permisos.

5. DATOS PERSONALES

- Nombre, teléfono y email de clientes no deben aparecer en logs ni en respuestas de endpoints públicos.

Formato de tu respuesta

Devuelve exactamente esto, sin introducción ni despedida:

CRÍTICO (no entregar así)

- [archivo:línea] Problema en una frase. Corrección en una frase.

IMPORTANTE (arreglar antes de cobrar)

- [archivo:línea] Problema. Corrección.

```
### MENOR (mejorable)
- [archivo:línea] Problema. Corrección.
```

```
### VERIFICADO OK
- Lista breve de lo que sí está bien.
```

Si una categoría está vacía, escribe "Ninguno". No inventes hallazgos para llenar categorías. Si no pudiste revisar algo por falta de acceso, dilo explícitamente en lugar de suponer.

Subagente 2: probador de flujos

`.claude/agents/probador-flujos.md`


```

---
name: probador-flujos
description: Prueba mentalmente los flujos completos de un sistema de
  reservas leyendo el código, y reporta en qué punto exacto se rompen.
  Cubre reserva feliz, doble reserva, cancelación, horarios fuera de
  atención y datos inválidos. Devuelve una tabla de casos con
  resultado esperado y resultado real.
tools: Read, Grep, Glob
model: inherit
---

```

Eres un probador de control de calidad para sistemas de reservas. Trabajas leyendo el código y siguiendo el flujo paso a paso, como si fueras el usuario. No escribes código ni corriges nada.

Casos que siempre pruebas

CAMINO FELIZ

1. Cliente entra, elige servicio, profesional, día y hora libre, llena sus datos y confirma. Debe quedar una reserva con status pending o confirmed y verse una pantalla de confirmación.

CASOS BORDE OBLIGATORIOS

2. Dos personas eligen el mismo slot al mismo tiempo. La segunda debe recibir un error claro, no una reserva duplicada.
3. Reserva con menos de la anticipación mínima permitida. Debe rechazarse.
4. Reserva fuera del horario de atención o en día cerrado. Debe rechazarse.
5. Reserva a más de 30 días vista. Debe rechazarse.
6. Teléfono vacío o con formato inválido. Debe rechazarse con mensaje visible al usuario.
7. Cancelación dentro del plazo permitido. Debe liberar el horario.
8. Cancelación fuera de plazo. Debe rechazarse.
9. Un servicio con duración distinta al resto. Los slots deben calcularse con SU duración, no con una fija.
10. Cambio de día en el calendario sin recargar. Los slots deben actualizarse.

Cómo trabajas

Para cada caso, sigue el código real: componente, handler, función de disponibilidad, consulta a la base, constraint. Di en qué archivo y línea se decide el resultado. Si no encuentras dónde se valida algo, ese es un hallazgo: significa que no se valida.

Formato de tu respuesta

Una tabla y nada más antes de ella:

```

| # | Caso | Esperado | Real | Dónde se decide | Estado |
|---|---|---|---|---|---|

```

Estado solo puede ser: PASA, FALLA o NO VALIDADO.

Después de la tabla, una sección "FALLAS EN ORDEN DE GRAVEDAD" con una línea por falla explicando la causa raíz, no el síntoma.

No propongas la solución completa. Solo di dónde está el problema.

Subagente 3: explorador de código

`.claude/agents/explorador.md`

```
---
name: explorador
description: Encuentra dónde vive algo en el proyecto y devuelve un
  mapa mínimo con rutas y números de línea, sin volcar el contenido de
  los archivos. Úsalo antes de modificar algo cuando no sabes dónde
  está, para no llenar la conversación principal de código.
tools: Read, Grep, Glob
model: inherit
---
```

Eres un explorador de código. Tu trabajo es localizar, no explicar y no modificar. Optimizas para que la respuesta sea lo más corta posible sin perder lo que hace falta para actuar.

Cómo trabajas

1. Empieza amplio con búsquedas por patrón, luego reduce.
2. Prueba varias formas de nombrar lo mismo: en español y en inglés, camelCase y snake_case, singular y plural. En este proyecto conviven ambos idiomas.
3. Abre un archivo completo solo si es imprescindible para responder.
4. Nunca pegues bloques largos de código en tu respuesta. Cita como máximo 3 líneas cuando la línea exacta sea el dato clave.

Formato de tu respuesta

DÓNDE ESTÁ

- ruta/exacta/archivo.ts:120 – qué hace en una frase

QUIÉN LO USA

- ruta/exacta/otro.tsx:45 – cómo lo llama

DE QUÉ DEPENDE

- ruta/exacta/tercero.ts – qué le aporta

PARA CAMBIARLO HAY QUE TOCAR

Lista ordenada de archivos, del más al menos importante, con una frase de qué cambio requiere cada uno.

Si no encuentras algo, dilo claramente y di qué términos buscaste. No inventes rutas. No supongas que un archivo existe sin verificarlo. Máximo 40 líneas de respuesta.

Subagente 4: redactor de contenido del cliente

`.claude/agents/redactor-cliente.md`

```
---
name: redactor-cliente
description: Escribe todos los textos visibles del sistema de reservas
  con la voz del negocio del cliente – títulos, instrucciones,
  mensajes de error, confirmaciones, correos y textos legales base.
  Español de LatAm, claro y sin jerga técnica.
tools: Read, Write, Edit
model: inherit
---
```

Eres redactor de producto para negocios locales en Latinoamérica.
Escribes los textos que ve el cliente final de una barbería, clínica,
taller o spa cuando reserva una cita.

Reglas de voz

- Español neutro de LatAm. Trato de "tú" con el cliente final.
- Frases cortas. Cero jerga técnica: nunca "error 500", "token inválido", "constraint", "payload", "slot".
- Un mensaje de error siempre dice tres cosas: qué pasó, por qué, y qué hacer ahora.
- Nada de humor forzado ni signos de exclamación en cadena.
- Sin emojis.
- Los montos en la moneda del país del negocio, con su formato local.
- Las horas en formato de 12 horas con am/pm si el país lo usa así.

Qué produces cuando te lo piden

1. Textos de la interfaz: títulos y subtítulos de cada paso, textos de botones, etiquetas y ayudas de formulario, estados vacíos.
2. Mensajes de error, uno por caso real: horario ya ocupado, fuera de horario, muy poca anticipación, datos incompletos, teléfono inválido, fallo de conexión.
3. Confirmación de reserva: pantalla y mensaje de texto o correo, con los datos que el cliente necesita recordar.
4. Recordatorio previo a la cita y aviso de cancelación.
5. Textos legales base: aviso de privacidad y términos de uso, adaptados al rubro. Marca con [REVISAR CON ABOGADO] toda parte que dependa de la legislación del país. Nunca afirmes que un texto legal cumple una normativa concreta.

Antes de escribir, confirma

Nombre comercial, rubro, ciudad y país, tono deseado (cercano o formal), política de cancelación real y si cobran anticipo. Si falta algún dato, pregúntalo. No inventes políticas del negocio.

Formato de tu respuesta

Agrupar por pantalla o por pieza, con la clave del texto y el texto final. Si te piden aplicarlo al código, editar solo cadenas de texto visibles: no cambiar nombres de variables, rutas ni lógica.

PARTE 7 — Orquestación: el flujo completo

Así se encadena todo para construir un sistema de reservas de principio a fin. Cada fase es una tarea cerrada, con su verificación.

Fase 0 — Preparar el terreno

Antes de pedir una sola línea de código: escribe `CLAUDE.md` con las reglas de negocio numeradas del cliente. Si no tienes las reglas, no tienes proyecto: llama al cliente y pregúntale horarios, duraciones, política de cancelación y anticipación mínima.

Fase 1 — Datos

Prompt directo: crear las tablas `services`, `staff`, `bookings` con sus migraciones. Una tarea, un archivo de migración, verificación: el SQL corre limpio.

Luego, tarea aparte: las políticas RLS. Nunca en el mismo prompt que las tablas.

Luego, tarea aparte: la protección contra doble reserva a nivel de base de datos.

Fase 2 — Lógica de disponibilidad

`lib/availability.ts`: dado un profesional, una fecha y un servicio, devolver los horarios libres respetando horario de atención, duración del servicio, reservas activas y anticipación mínima. Es la pieza más delicada del sistema: hazla sola y pruébala sola.

Al terminar, invoca al subagente **probador-flujos** limitado a los casos de disponibilidad.

Fase 3 — Interfaz del cliente

Componentes uno por uno: selector de servicio, selector de profesional, calendario, formulario, confirmación. Después de cada uno, verificación visual.

Aquí entra el subagente **redactor-cliente** para todos los textos, en una sola pasada, en lugar de improvisar copys pantalla por pantalla.

Fase 4 — Panel del negocio

Autenticación, lista de reservas del día, cambio de estado, bloqueo de horarios. El panel se hace después del flujo público, porque el flujo público es lo que genera el dinero del cliente.

Fase 5 — Auditoría

Invoca al subagente **revisor-seguridad**. Arregla lo CRÍTICO y lo IMPORTANTE. Vuelve a invocarlo. Solo entonces sigues.

Invoca al subagente **probador-flujos** completo. Arregla las fallas.

Fase 6 — Despliegue

Skill de despliegue: variables de entorno, build, dominio, comprobación post-despliegue con reserva real de prueba.

Fase 7 — El siguiente cliente

Skill `nuevo-nicho`. Aquí es donde el trabajo de las fases anteriores se paga solo: el segundo cliente cuesta una fracción del primero.

El patrón que se repite

```
Yo defino la tarea cerrada
-> Subagente explorador localiza si no sé dónde está
  -> Claude Code ejecuta el cambio
    -> Yo verifico contra el criterio de aceptación
      -> Limpio contexto
        -> Siguiendo tarea cerrada
```

Ese "limpio contexto" entre tareas es lo que separa una sesión barata de una cara. De eso trata la siguiente parte.

PARTE 8 — Ahorro de tokens: la sección más importante

Por qué el contexto largo cuesta y además empeora la calidad

Cada mensaje que mandas no viaja solo. Viaja con **toda la conversación previa**: el `CLAUDE.md`, cada archivo que se leyó, cada bloque de código que se pegó, cada log que copiaste, cada skill cargada. Eso significa dos cosas:

- Uno: cuesta dinero.** No pagas por tu pregunta de 20 palabras, pagas por las 80.000 palabras de contexto que arrastra.
 - Dos, y peor: baja la calidad.** Con el contexto muy lleno, la atención del modelo se reparte entre demasiadas cosas. Empieza a mezclar versiones viejas de un archivo con nuevas, a "recordar" decisiones que ya revertiste, a repetir errores ya corregidos. La señal clásica es que la IA vuelve a proponer algo que descartaste hace media hora.
- Regla que vale más que todo lo demás: **una conversación limpia y corta produce mejor código que una conversación larga y llena**. No aguantes una sesión de cuatro horas por comodidad.

Limpiar vs. compactar

Acción	Qué hace	Cuándo
Limpiar	Empieza de cero, contexto vacío	Cambias de tarea. El nuevo trabajo no necesita nada del anterior
Compactar	Resume lo hablado y sigue	Sigues en la MISMA tarea pero llevas mucho rato

Criterio práctico:

- Limpiar** al terminar cada tarea cerrada. Acabas de crear las tablas y ahora vas a hacer la UI: limpia. Lo que necesitas saber ya está en `CLAUDE.md` y en el código.
- Compacta** cuando llevas rato depurando un mismo bug y el hilo de razonamiento importa.
- Si dudas entre las dos: **limpia**. Casi siempre es la respuesta correcta, porque el estado real de tu proyecto vive en los archivos, no en el chat.

Antes de limpiar, haz una cosa: pide un resumen de tres líneas de lo hecho y lo pendiente, y pégalo como primer mensaje de la sesión nueva. Cuesta 60 tokens y te ahorra 20.000 de re-exploración.

Las 18 tácticas

Táctica 1 — Trabaja por tareas cerradas, no por sesiones eternas

Problema: una sola sesión que empieza en la base de datos y termina en el despliegue acumula todo el contexto de todo el proyecto.

Táctica: una tarea, una sesión. Cierra y limpia.

Antes: un chat de 5 horas donde al final la IA confunde el esquema viejo con el nuevo. **Después:** ocho sesiones de 20 minutos, cada una con un entregable verificado.

Táctica 2 — Delega la exploración a un subagente

Problema: "busca dónde se valida el teléfono" hace que se lean 15 archivos, y esos 15 archivos se quedan en tu contexto para siempre.

Táctica: manda al subagente `explorador`. Él lee 15 archivos en su contexto y te devuelve 6 líneas.

Antes:

```
Busca en todo el proyecto dónde se valida el teléfono del cliente.
```

Después:

```
Usa el subagente explorador para localizar dónde se valida el  
teléfono del cliente. Quiero solo rutas y líneas, no el código.
```

Táctica 3 — Pide diffs, no archivos completos

Problema: para cambiar tres líneas, te reimprime un archivo de 200. Pagas 200 líneas y encima tienes que revisarlas todas.

Táctica: pide explícitamente el cambio mínimo.

Antes:

```
Corrige la validación y muéstrame el archivo actualizado.
```

Después:

```
Corrige la validación editando el archivo directamente. Después  
muéstrame solo las líneas que cambiaste, con 2 líneas de contexto  
arriba y abajo. No reimprimas el archivo completo.
```

Táctica 4 — Nombra rutas exactas en lugar de pedir que busque

Problema: cada búsqueda ciega es lectura de archivos que se queda en contexto.

Táctica: si sabes dónde está, dilo. Ten a mano el mapa de carpetas.

Antes:

```
Cambia el color del botón de confirmar.
```

Después:

```
En components/booking/ConfirmButton.tsx, cambia el fondo del botón principal de bg-blue-600 a bg-neutral-900.
```

Táctica 5 — Pega la parte relevante del log, no el log entero

Problema: un stack trace de Next.js son 300 líneas, de las cuales 8 dicen algo.

Táctica: pega el mensaje de error, el archivo y la línea. Nada más.

Antes: 300 líneas de traza pegadas tal cual. **Después:**

```
Error en producción:  
TypeError: Cannot read properties of undefined (reading 'duration_min')  
en lib/availability.ts:34, dentro de getAvailableSlots.  
Pasa solo cuando el servicio no existe en la tabla services.
```

Táctica 6 — Usa archivos de referencia en lugar de repetir instrucciones

Problema: repites las mismas 40 líneas de reglas de estilo en cada prompt.

Táctica: escríbelas una vez en un archivo y refiérete a él por ruta.

Antes: pegar las convenciones completas en cada mensaje. **Después:**

```
Sigue las convenciones de docs/convenciones-ui.md. No hace falta que me las repitas, solo aplícalas.
```

Táctica 7 — El material largo va en `references/`, no en el cuerpo de la skill

Problema: el cuerpo del `SKILL.md` permanece en contexto durante los turnos siguientes una vez cargado. Cada línea se paga de forma recurrente.

Táctica: el `SKILL.md` es el índice de decisiones; el detalle vive en `references/` y se lee solo cuando el paso lo exige.

Antes: `SKILL.md` de 400 líneas con el checklist de seguridad completo dentro. **Después:** `SKILL.md` de 70 líneas que dice "en el paso 3, lee `#{CLAUDE_SKILL_DIR}/references/checklist-seguridad.md`".

Táctica 8 — No engordes el `CLAUDE.md`

Problema: el `CLAUDE.md` se carga siempre. Un `CLAUDE.md` de 600 líneas es un impuesto que pagas en cada turno de cada sesión, para siempre.

Táctica: deja ahí solo lo que aplica al 90% de los turnos. Lo demás, skill.

Antes: `CLAUDE.md` con el procedimiento completo de despliegue, los textos legales y el historial de decisiones.

Después: `CLAUDE.md` de 60 líneas: stack, mapa, reglas numeradas, comandos, prohibiciones.

Táctica 9 — Evita el ciclo "arréglalo" sin diagnóstico

Problema: decir "no funciona, arréglalo" hace que la IA adivine. Cada intento a ciegas cuesta tokens y suele agregar código nuevo encima del roto.

Táctica: primero diagnóstico, luego arreglo. Nunca los dos en el mismo mensaje.

Antes:

```
Sigue sin funcionar el calendario, arréglalo.
```

Después:

```
El calendario muestra slots ocupados como si estuvieran libres.  
Antes de tocar código: dime las 3 causas más probables, en qué  
archivo y línea se decide eso, y qué comprobación distingue entre  
ellas. No edites nada todavía.
```

Táctica 10 — Pide el plan antes del código en cambios grandes

Problema: un cambio grande mal entendido genera veinte archivos que hay que tirar. Pagaste por generarlos y pagas por revisarlos.

Táctica: plan primero, aprobación, luego ejecución.

Antes:

```
Agrega el sistema de anticipos con Stripe.
```

Después:

```
Quiero agregar cobro de anticipo. Antes de escribir código: lista  
los archivos que tocarías, qué cambia en cada uno y qué decisiones  
necesitas de mí. Máximo 15 líneas. Espera mi OK.
```

Táctica 11 — Una tarea por mensaje

Problema: pedir cinco cosas juntas hace que se hagan tres bien, una a medias y una mal, y el desenredo cuesta más que hacerlas por separado.

Táctica: un objetivo por mensaje, verificado antes de seguir.

Antes:

```
Crea la tabla, las políticas RLS, el endpoint, el componente y el  
correo de confirmación.
```

Después: cinco mensajes, cada uno con su criterio de aceptación.

Táctica 12 — No releas archivos que ya están en contexto

Problema: pedir "léeme otra vez el archivo X" duplica su costo, y ahora hay dos versiones del mismo archivo compitiendo en contexto.

Táctica: si ya se leyó y no lo has editado fuera, refiérete a él por nombre.

Antes:

```
Vuelve a leer lib/availability.ts y dime qué hace la línea 34.
```

Después:

```
En lib/availability.ts que ya leíste, ¿qué asume la línea 34 sobre el objeto service?
```

Táctica 13 — Limita el alcance de la lectura por adelantado

Problema: "revisa el proyecto" es una invitación a leer todo.

Táctica: acota carpeta, archivo o rango de líneas.

Antes:

```
Revisa el proyecto y dime si hay problemas de seguridad.
```

Después:

```
Revisa únicamente supabase/migrations/ y dime qué tablas no tienen RLS habilitado. No leas app/ ni components/.
```

Táctica 14 — Prohíbe explícitamente el trabajo extra

Problema: la IA "aprovecha" para refactorizar, agregar comentarios, reorganizar imports o mejorar cosas que no pediste. Todo eso son tokens de salida que pagas y cambios que tienes que revisar.

Táctica: cierra cada prompt con qué NO hacer.

Antes:

```
Arregla el bug de la zona horaria.
```

Después:

```
Arregla el bug de la zona horaria en lib/availability.ts.  
NO refactorices otras funciones, NO agregues comentarios nuevos,  
NO reordenes imports, NO toques otros archivos. Cambio mínimo.
```

Táctica 15 — Corta las respuestas largas antes de que ocurran

Problema: te explica en diez párrafos algo que se resuelve en tres líneas, y esa explicación se queda en el contexto encareciendo todos los mensajes siguientes.

Táctica: pon un techo de longitud.

Antes:

```
Explícame cómo funciona el cálculo de disponibilidad.
```

Después:

```
Explícame el cálculo de disponibilidad en máximo 8 líneas, sin código y sin repetir lo que ya dice el archivo.
```

Táctica 16 — Trabaja contra el archivo, no contra el chat

Problema: usar el chat como memoria del proyecto. "¿Te acuerdas de que dijimos que la anticipación era 2 horas?" Si vive solo en el chat, se pierde al limpiar.

Táctica: toda decisión que valga la pena recordar se escribe en `CLAUDE.md` o en el código en el momento en que se toma.

Antes: decisiones dispersas en 200 mensajes. **Después:** `CLAUDE.md` regla 4: "No se aceptan reservas con menos de 2 horas de anticipación."

Táctica 17 — No pidas alternativas cuando ya decidiste

Problema: "dame 3 opciones" triplica la salida, y luego descartas dos.

Táctica: pide opciones solo cuando de verdad vas a elegir, y pídelas en una línea cada una.

Antes:

```
Dame 3 formas de evitar la doble reserva, con el código de cada una.
```

Después:

```
Dame 3 formas de evitar la doble reserva, una línea por opción con su principal desventaja. Sin código. Después te digo cuál.
```

Táctica 18 — Cierra la sesión cuando la calidad baje, no cuando te canses

Problema: aguantas la sesión "porque ya casi", y las últimas dos horas producen el peor código del día.

Táctica: hay tres señales de que toca limpiar ya: la IA repite un error que ya corregiste, propone algo que descartaste, o confunde el estado actual de un archivo. En cuanto veas una, cierra.

Antes: tres horas peleando con un contexto contaminado. **Después:** resumen de tres líneas, sesión nueva, y el problema se resuelve en dos mensajes.

Tabla rápida de referencia

Situación	Qué hacer
No sé dónde está algo	Subagente explorador
Cambio pequeño	Nombra la ruta exacta y pide diff
Cambio grande	Plan primero, aprobación, luego código
Falla algo	Diagnóstico primero, arreglo después
Terminé una tarea	Resumen de 3 líneas y limpiar
Llevo rato en el mismo bug	Compactar
Material largo repetido	Archivo de referencia
Procedimiento repetido	Skill con <code>references/</code>
Regla permanente	CLAUDE.md numerada

PARTE 9 — Protocolo de depuración en 6 pasos

Este protocolo es la versión larga y explicada. La skill `/arreglar-errores` trae la versión operativa de los mismos seis pasos, con la nomenclatura que usa el resto del paquete: reproducir, aislar, formular una hipótesis, hacer el cambio mínimo, verificar y registrar la causa. Si están en desacuerdo, manda la skill.

Cuando la IA se atasca en un bug, el instinto es decir "sigue intentando". Es la peor decisión posible: cada intento a ciegas agrega código nuevo sobre el roto y llena el contexto. Usa esto en su lugar.

Paso 1 — Congelar el síntoma

No se puede arreglar lo que no está descrito con precisión.

Vamos a depurar sin tocar código todavía.

SÍNTOMA: [qué ves exactamente]

DÓNDE: [ruta o pantalla]

CUÁNDO: [pasos exactos para reproducirlo]

QUÉ ESPERABA: [comportamiento correcto]

QUÉ PASA: [comportamiento real]

¿SIEMPRE?: [siempre / solo a veces / solo con estos datos]

Reformula el problema en una sola frase para confirmar que entendemos lo mismo. No propongamos soluciones aún.

Paso 2 — Hipótesis, no soluciones

Dame las 3 causas más probables de este síntoma, ordenadas por probabilidad. Para cada una: en qué archivo y línea se decidiría, y qué comprobación concreta la confirma o la descarta.

No edites ningún archivo. Máximo 12 líneas en total.

Paso 3 — Aislar

Vamos con la hipótesis 1. Dime la forma más barata de comprobarla: un `console.log`, una consulta SQL, o abrir un archivo concreto.

Quiero una sola comprobación, la que más información dé. Dime exactamente qué debo ver si la hipótesis es correcta y qué debo ver si es incorrecta.

Paso 4 — Confirmar con el dato real

Resultado de la comprobación: [pega SÓLO la salida relevante]

¿Confirma o descarta la hipótesis 1? Responde en 3 líneas. Si la descarta, pasamos a la hipótesis 2 sin escribir código.

Paso 5 — Arreglar lo mínimo

Confirmado: la causa es [causa].

Aplica el cambio MÍNIMO que la corrige, en [archivo]. Reglas:

- No refactorices nada más
- No agregues librerías
- No cambies otros archivos
- No agregues comentarios nuevos

Después muéstrame solo las líneas modificadas y explica en 2 líneas por qué esto lo corrige.

Paso 6 — Blindar y cerrar

El bug está corregido. Ahora:

1. ¿Existe el mismo error en otro punto del proyecto? Búscalo solo en [carpeta acotada] y dame rutas, sin arreglarlo todavía.
2. ¿Qué regla debería agregar a CLAUDE.md para que no vuelva a pasar? Redáctala en una línea, numerada.
3. ¿Se puede impedir a nivel de base de datos o de tipos, en lugar de con una validación más?

Regla de oro del protocolo: si después del paso 4 sigues sin causa confirmada, no insistas en la misma sesión. Limpia el contexto, escribe el síntoma desde cero y empieza otra vez en el paso 1. Una sesión contaminada no encuentra bugs.

PARTE 10 — Biblioteca de 20 prompts listos para copiar

Ajusta los datos entre corchetes. Todo lo demás está pensado para no dejar huecos que la IA rellene inventando.

1. Esquema de base de datos

Crea el esquema base del sistema de reservas para [tipo de negocio].

Tablas: services (id, name, duration_min, price_cents, active), staff (id, full_name, active), bookings (id, service_id, staff_id, customer_name, customer_phone, customer_email, starts_at, ends_at, status, notes, created_at).

Reglas: PK uuid con gen_random_uuid(). Todas las fechas en timestampz. status como enum: pending, confirmed, cancelled, no_show. Índice sobre (staff_id, starts_at).

Entrega: un solo archivo en supabase/migrations/ con nombre por fecha. Nada de código de aplicación.

ACEPTACIÓN: corre limpio en un Postgres vacío.

NO HAGAS: políticas RLS (van aparte), datos de ejemplo.

2. Políticas RLS

Escribe las políticas RLS para services, staff y bookings.

Modelo de acceso:

- Cualquiera (anónimo) puede LEER services y staff donde active = true.
- Cualquiera puede INSERTAR en bookings, pero solo con status 'pending'.
- Nadie anónimo puede LEER, ACTUALIZAR ni BORRAR bookings de otros.
- Los usuarios autenticados del negocio pueden hacer todo sobre bookings.

Entrega: una migración SQL nueva. Habilita RLS explícitamente en las tres tablas. Para cada política, un comentario de una línea diciendo qué permite y a quién.

ACEPTACIÓN: ninguna política usa `true` como condición para leer bookings.

NO HAGAS: no uses la service_role key como solución.

3. Protección contra doble reserva

Impide a nivel de base de datos que dos reservas ACTIVAS del mismo profesional se solapen.

Contexto: bookings tiene staff_id, starts_at, ends_at, status.
Cancelled y no_show NO bloquean el horario.

Evalúa usar btree_gist con una constraint EXCLUDE sobre
tstzrange(starts_at, ends_at) filtrada por status activo. Si no es viable, explica por qué y propón la alternativa ANTES de escribir.

Entrega: migración SQL + 5 líneas sobre qué error verá la app cuando salte la constraint y cómo debe traducirse a un mensaje al usuario.
ACEPTACIÓN: dos confirmadas solapadas del mismo staff fallan; una cancelada solapada pasa; dos profesionales distintos a la misma hora pasan.

4. Cálculo de disponibilidad

Crea lib/availability.ts con la función:
getAvailableSlots(staffId: string, date: Date, serviceId: string):
Promise<string[]>

Debe respetar, en este orden:

1. Horario de atención: [09:00-20:00], cerrado [domingo]
2. Duración real del servicio consultada en la tabla services
3. Reservas existentes con status pending o confirmed
4. Anticipación mínima de [2] horas desde ahora
5. Máximo [30] días a futuro

Devuelve horas ISO en UTC, ordenadas. Array vacío si no hay nada.
Entrega: un solo archivo, TypeScript estricto, sin librerías nuevas.
ACEPTACIÓN: un servicio de 30 min y otro de 60 min generan rejillas distintas para el mismo día.
NO HAGAS: no asumas duración fija, no uses Date local sin zona.

5. Calendario de disponibilidad (UI)

Crea components/booking/AvailabilityCalendar.tsx.

Props: staffId: string, serviceId: string, date: Date,
onSelect: (iso: string) => void.

Los datos vienen de getAvailableSlots de lib/availability.ts.

Requisitos:

- Solo Tailwind, sin librerías de calendario.
- Grid de 3 columnas desde 360px, 6 en escritorio.
- Solo se renderizan los slots libres.
- Estado vacío: "No hay horarios disponibles para este día".
- Slot seleccionado: fondo negro, texto blanco.
- Estado de carga visible mientras se consultan los datos.

Entrega: un solo archivo.

NO HAGAS: no instales dependencias, no metas fetch dentro del componente, no modifiques lib/availability.ts.

6. Formulario de datos del cliente

Crea el formulario final del flujo de reserva.

Campos: nombre completo (requerido), teléfono (requerido, formato de [país]), email (opcional), notas (opcional, máx 300 caracteres).

Requisitos:

- Validación en cliente para experiencia, PERO la validación real debe existir también en el servidor. Indica dónde la pusiste.
- Mensajes de error en español, debajo de cada campo, diciendo qué hacer para corregir.
- Botón deshabilitado mientras se envía, para evitar doble envío.

Entrega: componente + la validación de servidor.

ACEPTACIÓN: enviar con teléfono vacío no crea reserva ni siquiera si se manipula el formulario desde el navegador.

7. Confirmación de reserva

Crea la pantalla de confirmación tras reservar.

Debe mostrar: nombre del negocio, servicio, profesional, fecha y hora en zona [America/Mexico_City] en formato legible, duración, precio, dirección, y la política de cancelación ([hasta 4 horas antes]).

También un botón para cancelar que lleve a la ruta de cancelación con el identificador de la reserva.

Entrega: página + componente. Textos en español de [país], trato de "tú", sin jerga técnica.

NO HAGAS: no muestres ids internos ni datos de otras reservas.

8. Panel del negocio

Crea el dashboard del negocio en `app/(admin)/dashboard`.

Debe mostrar:

1. Reservas de HOY ordenadas por hora, con nombre, teléfono, servicio, profesional y estado.
2. Contadores del día: total, confirmadas, canceladas, no_show.
3. Acción por reserva para cambiar estado entre confirmed, cancelled y no_show.
4. Selector de fecha para ver otros días.

Requisitos: la sesión se verifica EN EL SERVIDOR antes de renderizar cualquier dato. Si no hay sesión, redirige a login.

Entrega: primero la lista de archivos que vas a crear y qué hace cada uno. Espera mi OK antes de escribir código.

9. Bloqueo manual de horarios

El negocio necesita bloquear horarios sin que sean reservas: comida, vacaciones, cierre por evento.

Diseña la solución más simple que respete la protección de solapamiento que ya existe. Compara en 3 líneas cada opción: (a) tabla `blocks` aparte, (b) reservas internas con un status especial. Recomienda una y explica por qué.

No escribas código hasta que yo elija.

10. Recordatorio antes de la cita

Quiero que el cliente reciba un recordatorio [24] horas antes de su cita.

Antes de escribir código, dime:

1. Qué opciones hay para ejecutar algo de forma programada con el stack actual (Next.js en Vercel + Supabase).
2. Qué necesita cada opción en configuración y qué límites tiene.
3. Cuál recomiendas para un negocio con menos de [50] reservas diarias y por qué.

Máximo 15 líneas. Sin código todavía.

11. Adaptar el proyecto a otro nicho

Adapta este sistema de reservas de [barbería] a [clínica dental].

Cambios reales:

1. "[barbero]" pasa a "[doctor]" en toda la interfaz visible
2. Duraciones por tratamiento: [30 y 60 min]
3. Campo nuevo: motivo de consulta, texto libre, máx 300 caracteres
4. Aviso de manejo de datos en el formulario

Entrega: primero la lista de archivos a tocar y el cambio en cada uno. Espera mi OK.

ACEPTACIÓN: `npm run build` pasa y no queda la palabra "[barbero]" en app/ ni components/.

NO HAGAS: no renombres tablas ni columnas, no toques RLS ni auth.

12. Auditoría de seguridad previa a la entrega

Usa el subagente revisor-seguridad para auditar el proyecto completo antes de entregarlo al cliente.

Quiero su reporte tal cual, sin que agregues análisis propio ni corrijas nada todavía. Después de leerlo yo decido qué se arregla.

13. Prueba de flujos completa

Usa el subagente probador-flujos para revisar los 10 casos del flujo de reserva.

Devuélveme solo su tabla de resultados y la lista de fallas por gravedad. No arregles nada en este turno.

14. Preparar el despliegue

Prepara el proyecto para desplegar en Vercel.

1. Lista TODAS las variables de entorno que la app necesita, separando cuáles son públicas (NEXT_PUBLIC_) y cuáles no deben salir nunca del servidor.
2. Confirma que .env y .env.local están en .gitignore.
3. Ejecuta `npm run build` y reporta cualquier error o advertencia de tipos.
4. Dime qué comprobación manual debo hacer después de desplegar, en 5 pasos concretos.

NO HAGAS: no despliegues nada, no cambies configuración sin avisarme antes.

15. Comprobación después de desplegar

Ya está desplegado en [url]. Dame una lista de verificación de 10 puntos que yo pueda ejecutar a mano en 10 minutos para confirmar que el sistema funciona en producción.

Cada punto: qué hacer, qué debo ver si está bien, qué significa si no lo veo. En una línea cada uno.

16. Textos legales

Usa el subagente redactor-cliente para escribir el aviso de privacidad y los términos de uso de [nombre del negocio], un [rubro] en [ciudad, país].

Datos que recolectamos: nombre, teléfono, email opcional, notas de la cita. Política de cancelación: [hasta 4 horas antes].

Marca con [REVISAR CON ABOGADO] toda parte que dependa de la legislación local. No afirmes que el texto cumple ninguna normativa concreta.

17. Todos los mensajes del sistema

Usa el subagente redactor-cliente para escribir TODOS los textos visibles del sistema, de una sola vez.

Necesito: títulos de cada paso del flujo, textos de botones, etiquetas de formulario, estados vacíos, y un mensaje de error para cada uno de estos casos: horario ya ocupado, fuera de horario, poca anticipación, más de 30 días vista, teléfono inválido, datos incompletos, fallo de conexión.

Formato: tabla con clave del texto y texto final. Español de [país], trato de "tú".

18. Revisión de rendimiento del calendario

El calendario tarda en cargar. Antes de optimizar nada:

1. Dime cuántas consultas a la base de datos se hacen para pintar un día completo, y dónde están.
2. Dime si alguna se repite innecesariamente.
3. Ordena por impacto las 3 mejoras posibles, con el esfuerzo de cada una.

Máximo 12 líneas. No cambies código todavía. Revisa solo lib/availability.ts y components/booking/.

19. Datos de prueba realistas

Genera datos de prueba para poder demostrar el sistema al cliente.

Necesito: [3] profesionales, [6] servicios con duraciones y precios distintos y realistas para [rubro] en [país], y [25] reservas repartidas entre ayer, hoy y los próximos 7 días, con mezcla de estados confirmed, pending, cancelled y no_show.

Entrega: un archivo SQL de inserción aparte, claramente marcado como datos de prueba, en una carpeta seed/ y NO en migrations/.
ACEPTACIÓN: ninguna reserva generada viola la restricción de solapamiento.

20. Traspaso al cliente

El proyecto está terminado. Genera el documento de entrega para el dueño del negocio, que no es técnico.

Debe incluir: cómo entra a su panel, cómo cambia el estado de una reserva, cómo bloquea un horario, cómo agrega o desactiva un servicio, qué hacer si un cliente llama para cancelar, y a quién escribir si algo falla.

Español simple, trato de "tú", sin jerga técnica, sin capturas.
Máximo 2 páginas. Guárdalo en docs/manual-cliente.md.

PARTE 11 — Los 12 errores más caros al dirigir la IA

1. Pedir el sistema completo de una vez

Corrección: trabaja en tareas cerradas y verificables. Si no puedes decir en una frase cómo comprobarás que quedó bien, la tarea es demasiado grande.

2. No tener CLAUDE.md, o tenerlo desactualizado

Corrección: escríbelo antes del primer prompt de código y actualízalo en el momento en que tomas una decisión, no "después".

3. Meter en CLAUDE.md lo que debería ser una skill

Corrección: el `CLAUDE.md` se carga siempre y lo pagas en cada turno. Si algo aplica solo a un tipo de tarea, sácalo a una skill.

4. Escribir descriptions vagas en las skills

Corrección: caso de uso primero, frases disparadoras reales, errores de escritura comunes, y qué NO cubre. Recuerda el corte de 1.536 caracteres.

5. Poner todo el contenido dentro del SKILL.md

Corrección: el cuerpo permanece en contexto turno tras turno. `SKILL.md` corto como índice, detalle en `references/`.

6. Decir "arréglalo" sin diagnóstico

Corrección: protocolo de 6 pasos. Hipótesis antes que parches. Un intento a ciegas cuesta más que dos minutos de diagnóstico.

7. Aceptar "ya está listo" sin verificar

Corrección: define el criterio de aceptación ANTES de que empiece, y compruébalo tú. "Listo" solo significa lo que tú definiste que significa.

8. Dejar la validación solo en el frontend

Corrección: toda regla de negocio que importe dinero o integridad va en el servidor o en la base de datos. El navegador es del usuario, no tuyo.

9. Sesiones eternas

Corrección: limpia al cerrar cada tarea. Si la IA repite un error corregido o propone algo descartado, cierra en ese momento.

10. Pegar logs completos y archivos enteros

Corrección: pega el error, el archivo y la línea. Pide diffs, no reimpresiones. Nombra rutas exactas en vez de pedir búsquedas.

11. No usar subagentes para explorar

Corrección: la exploración es lectura masiva con resultado pequeño. Es exactamente para lo que sirve un subagente con su propia ventana de contexto.

12. No prohibir el trabajo extra

Corrección: cierra cada prompt con qué NO hacer. Sin eso, la IA refactoriza, renombra y "mejora" cosas que funcionaban, y tú pagas por revisar cambios que nunca pediste.

Cierre

Si te quedas con cinco ideas de todo el manual, que sean estas:

1. **La IA falla por cuatro razones:** falta contexto, la instrucción es ambigua, la tarea es demasiado grande o no hay criterio de terminado. Todas se corrigen desde tu lado.
2. **Un prompt completo tiene seis partes**, y la que casi nadie escribe —qué NO hacer— es la que más dinero salva.
3. **Lo permanente va en `CLAUDE.md`, lo repetible en una skill, lo largo en `references/`**. Cada cosa en su lugar es dinero ahorrado en cada turno.
4. **Los subagentes existen para leer mucho y devolver poco.** Úsalos siempre que la tarea sea explorar, revisar o probar.
5. **El contexto limpio es una decisión, no un accidente.** Tareas cerradas, resumen de tres líneas, sesión nueva.

Nada de esto se aprende leyendo. Elige un proyecto que ya tengas, escríbele el `CLAUDE.md` esta semana, saca una skill de lo que más repites y copia los cuatro subagentes. Con eso solo, tu segundo cliente te va a costar la mitad que el primero.

Dasvo IA Builders